Programming with Python

This comprehensive guide will cover various fundamental topics in Python programming. It aims to provide detailed explanations, examples, and code snippets to help you understand and master these concepts. The topics covered in this guide include:

Official Documentation

- Python Documentation
- Python Tutorial

Table of Contents

Let's get started!

Variables

In Python, variables are used to store data. They act as containers that hold values of different types, such as numbers, strings, or more complex data structures. To assign a value to a variable, you can use the assignment operator `=`. Here's an example:

```
x = 10
y = "Hello, World!"
```

Variable names can contain letters, numbers, and underscores, but they cannot start with a number. Python is case-sensitive, so `my_variable`, `My_Variable`, and `MY_VARIABLE` are considered different variables.

Data Types

Numeric Types

Python supports several numeric types, including integers (`int`), floating-point numbers (`float`), and complex numbers (`complex`). Here's an example:

```
# Integer
x = 10

# Floating-point number
y = 3.14

# Complex number
z = 2 + 3j
```

Strings

Strings are used to represent text data in Python. You can define strings using either single quotes (`'`) or double quotes (`"`). Here's an example:

```
message = "Hello, World!"
```

You can perform various operations on strings, such as concatenation (`+`), slicing, and accessing individual characters.

Booleans

Booleans represent the truth values `True` or `False`. They are often used in conditions and logical operations. Here's an example:

```
is_valid = True
is_invalid = False
```

Lists, Sets, and Dictionaries

Lists

Lists are mutable, ordered collections that can store elements of different types. Here are some common operations you can perform on lists:

```
fruits = ["apple", "banana", "cherry"]

# Accessing elements
print(fruits[0])    # Output: apple

# Modifying elements
fruits[1] = "pear"
print(fruits)       # Output: ['apple', 'pear', 'cherry']

# Adding elements
fruits.append("orange")
print(fruits)       # Output: ['apple', 'pear', 'cherry', 'orange']

# Removing elements
fruits.remove("apple")
print(fruits)       # Output: ['pear', 'cherry', 'orange']
```

Sets

Sets are mutable, unordered collections of unique elements. Here are some common operations you can perform on sets:

```
colors = {"red", "green", "blue"}

# Adding elements
colors.add("yellow")
print(colors)       # Output: {'red', 'green', 'blue', 'yellow'}
```

```
# Removing elements
colors.remove("red")
print(colors)       # Output: {'green', 'blue', 'yellow'}
```

Dictionaries

Dictionaries are mutable, unordered collections of key-value pairs. Here are some common operations you can perform on dictionaries:

```
person = {
    "name": "John",
    "age": 30,
    "city": "New York"
}

# Accessing values
print(person["name"])   # Output: John

# Modifying values
person["age"] = 31
print(person)           # Output: {'name': 'John', 'age': 31, 'city':
'New York'}

# Adding new key-value pairs
person["gender"] = "Male"
print(person)           # Output: {'name':'John', 'age': 31, 'city':
'New York', 'gender': 'Male'}

# Removing key-value pairs
del person["city"]
print(person)           # Output: {'name': 'John', 'age': 31, 'gender':
'Male'}
```

Operators

Arithmetic Operations

Python supports various arithmetic operations, including addition (+), subtraction (-), multiplication (*), division (/), and exponentiation (**). Here are a few examples:

```
a = 5
b = 2

sum = a + b          # Addition
difference = a - b   # Subtraction
product = a * b      # Multiplication
quotient = a / b     # Division
exponent = a ** b    # Exponentiation
```

Comparison Operators

Python provides comparison operators to compare values. These include equal to (`==`), not equal to (`!=`), greater than (`>`), less than (`<`), greater than or equal to (`>=`), and less than or equal to (`<=`).

```
a = 5
b = 2

is_equal = a == b           # Equal to
is_not_equal = a != b       # Not equal to
is_greater = a > b          # Greater than
is_less = a < b             # Less than
is_greater_equal = a >= b   # Greater than or equal to
is_less_equal = a <= b      # Less than or equal to
```

Logical Operators

Python provides logical operators to combine multiple conditions. These include `and`, `or`, and `not`. Here's an example:

```
a = 5
b = 2

# Logical AND
result_and = (a > 0) and (b < 0)

# Logical OR
result_or = (a > 0) or (b < 0)

# Logical NOT
result_not = not (a > 0)
```

Assignment Operators

Python provides shorthand assignment operators to perform arithmetic operations and assign the result back to a variable. These include `+=`, `-=`, `*=`, `/=`, and `**=`. Here's an example:

```
a = 5

a += 2   # Equivalent to: a = a + 2
a -= 2   # Equivalent to: a = a - 2
a *= 2   # Equivalent to: a = a * 2
a /= 2   # Equivalent to: a = a / 2
a **= 2  # Equivalent to: a = a ** 2
```

Identity Operators

Python provides identity operators to compare the memory location of two objects. These include `is` and `is not`. Here's an example:

```
a = [1, 2, 3]
b = [1, 2, 3]

# Check if a and b refer to the same object
is_same = a is b
# Check if a and b refer to different objects
is_different = a is not b
```

Membership Operators

Python provides membership operators to check if a value is present in a sequence (such as a list, set, or dictionary). These include `in` and `not in`. Here's an example:

```
fruits = ["apple", "banana", "cherry"]

# Check if "apple" is in the list
is_present = "apple" in fruits
# Check if "pear" is not in the list
is_absent = "pear" not in fruits
```

Conditions and Conditional Statements

If Statements

If statements allow you to execute different blocks of code based on certain conditions. Here's an example:

```
age = 18

if age >= 18:
    print("You are an adult.")
elif age >= 13:
    print("You are a teenager.")
else:
    print("You are a child.")
```

Loops: while and for, continue, break

while Loop

The `while` loop repeatedly executes a block of code as long as a given condition is true. Here's an example:

```
count = 0

while count < 5:
    print(count)
    count += 1
```

for Loop

The `for` loop iterates over a sequence of elements. Here's an example:

```
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(fruit)
```

continue and break Statements

- The `continue` statement is used to skip the current iteration and move to the next one in the loop.
- The `break` statement is used to exit the loop prematurely. It terminates the loop and continues with the next statement after the loop.

```
for i in range(10):
    if i % 2 == 0:
        continue    # Skip even numbers
    elif i == 7:
        break       # Exit the loop when i is 7
    print(i)
```

Functions

Defining Functions

Functions in Python allow you to encapsulate reusable blocks of code. They take input arguments (if any) and can return a value (or not). You can define your own functions using the `def` keyword. Here's an example of a function that calculates the sum of two numbers:

```
def add_numbers(a, b):
    sum = a + b
    return sum
```

You can call this function by passing two numbers as arguments:

```
result = add_numbers(3, 4)
print(result)  # Output: 7
```

External Libraries

Installing External Libraries

To use external libraries in Python, you need to install them. The most common way to install libraries is using the package manager `pip`. For example, to install the `requests` library, you can run the following command in your terminal:

```
pip install requests
```

Importing External Libraries

Once a library is installed, you can import it into your Python code using the `import` statement. Here's an example:

```
import requests

response = requests.get("https://www.example.com")
print(response.status_code)
```

Some libraries may have submodules or functions that you can import individually. For example:

```
from math import sqrt

result = sqrt(16)
print(result)  # Output: 4.0
```

Python comes with a rich ecosystem of libraries that can help you perform various tasks, such as data analysis, web scraping, machine learning, and more. Explore the Python Package Index (PyPI) to discover libraries that suit your needs. Some libraries come pre-installed with Python, while others need to be installed separately using `pip`.

Popular Libraries

Some popular libraries include:

- `numpy` for numerical computing.

- `pandas` for data manipulation and analysis.
- `matplotlib` for data visualization.
- `scikit-learn` for machine learning.
- `flask` for web development.

Class and Object

Python is an object-oriented programming language, which means it supports classes and objects. A class is a blueprint for creating objects, while an object is an instance of a class. Classes can have attributes (variables) and methods (functions).

Here's an example of defining a class and creating objects:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def greet(self):
        print(f"Hello, my name is {self.name} and I am {self.age} years
old.")

# Create objects of the Person class
person1 = Person("Alice", 30)
person2 = Person("Bob", 25)

# Call the greet method on the objects
person1.greet()
person2.greet()
```

In this example, the `Person` class has attributes `name` and `age`, and a method `greet` that prints a greeting message. We create two objects of the `Person` class and call the `greet` method on each object.

Basics of Terminal Commands

Running Python Files

To run a Python file, open your terminal or command prompt and navigate to the directory where the file is located. Then, use the `python` command followed by the file name and its extension. For example:

```
python my_script.py
```

Navigating Directories

- `cd directory_name` - Change to a specific directory.
- `cd ..` - Move up to the parent directory.
- `cd /` - Move to the root directory.

Listing Files and Directories

- `ls` - List files and directories in the current directory.
- `ls -l` - List files and directories with detailed information.
- `ls -a` - List all files and directories, including hidden ones.

Creating and Deleting Files/Directories

- `touch file_name` - Create an empty file.
- `mkdir directory_name` - Create a new directory.
- `rm file_name` - Delete a file.
- `rmdir directory_name` - Delete an empty directory.

Conclusion

This guide has introduced you to the basics of Python programming, covering topics such as variables, data types, lists, sets, dictionaries, operators, conditions, loops, functions, external libraries, and terminal commands. By mastering these concepts, you'll be well-equipped to write Python code for a wide range of applications. Keep practicing, experimenting, and learning to enhance your Python skills further. Good luck on your programming journey!